

GPU Memory Optimization: Tiling and Recomputation in Modern Deep Learning Systems

ZHENG, Bowen

A Thesis Submitted in Partial Fulfilment
of the Requirements for the Degree of
Final Report
in
Operating Systems

The Chinese University of Hong Kong, Shenzhen
April 26, 2026

Thesis Assessment Committee

Professor CHUNG, Yehching (Instructor)

LIU, Yuxuan (Teaching Assistant)

Abstract

Training Transformer-based large language models is bottlenecked by GPU memory. The self-attention operator computes an intermediate score matrix $S = QK^\top$ whose size grows quadratically with the sequence length N , while modern accelerators have a very wide gap between fast on-chip memory and slow off-chip memory. On NVIDIA’s H100, the per-SM shared memory is about 228 KB, while the HBM3 main memory is 80 GB at 3.35 TB/s. The capacity ratio is roughly 3.5×10^5 . For any sequence length above a few thousand, the working set of attention does not fit in fast memory and must be staged through HBM, leaving the tensor cores stalled on memory traffic.

This report surveys two families of techniques that address this problem on the H100. The first family is tiling, used in FlashAttention [6] and FlashAttention-3 [24]. The score matrix is decomposed into blocks small enough to live in shared memory, fusing the three kernels of standard attention into one and removing the dominant HBM round-trips. FlashAttention-3 layers a producer/-consumer pipeline with a circular shared-memory buffer on top of the tile loop, which keeps three different hardware units (the TMA, the tensor cores, and the special-function unit) busy at the same time.

The second family is recomputation, used in the FlashAttention backward pass and in gradient checkpointing [2]. Intermediate values that would otherwise occupy memory are dropped during the forward pass and regenerated when the

backward pass needs them. FlashAttention’s backward applies this idea at the operator level by dropping S and P and reconstructing them on-chip. Gradient checkpointing applies the same idea at the network level by storing only every $\sqrt{L}$ -th layer’s activations.

Both families correspond closely to ideas from operating systems. Tiling is cache blocking, the FlashAttention-3 pipeline is producer/consumer with a bounded queue, and gradient checkpointing is demand paging applied at layer granularity. The report works through each technique in detail and discusses what the analogy with operating systems explains and what it does not.

Acknowledgement

I would like to thank Professor Chung for an excellent course and for the guidance throughout the semester. I am also grateful to Mr. Liu for the help and feedback as the teaching assistant.

Contents

Abstract	i
Acknowledgement	iii
List of Figures	vii
List of Tables	ix
1 Introduction	1
1.1 The Memory Wall in Modern Deep Learning	1
1.2 Connection to Operating Systems	2
1.3 Organization	4
2 Preliminary	5
2.1 Why Different Memories Have Different Speeds	5
2.2 The H100 Memory Hierarchy	6
2.3 The Tensor Memory Accelerator (TMA)	8
2.4 Self-Attention and Its Working Set	9
2.5 Two Solution Strategies	11
3 Tiling: FlashAttention to FlashAttention-3	12
3.1 Two Layers of Benefit	12

3.2	Standard Attention: Three Separate Kernels	13
3.3	FlashAttention: Tiling and Online Softmax	14
3.3.1	Block decomposition of S	14
3.3.2	The forward algorithm	14
3.3.3	Online softmax	15
3.3.4	Numerical example	17
3.3.5	Complexity and intermediate result on A100	17
3.4	FlashAttention-3: Three-Way Asynchronous Pipeline	18
3.4.1	The throughput gap	18
3.4.2	Warp specialization and the producer/consumer pattern	18
3.4.3	The OS pattern, made concrete	20
3.5	Performance Comparison	20
3.6	The OS Analogy in One Picture	21

4 Recomputation: FlashAttention Backward and Gradient Checkpointing 23

4.1	The Compute-for-Memory Trade-off	24
4.2	FlashAttention Backward as Recomputation	25
4.2.1	The standard backward	25
4.2.2	FlashAttention’s backward	25
4.2.3	Empirical numbers	26
4.3	Why Activations Dominate Training Memory	27
4.4	Gradient Checkpointing: The $\sqrt{L}$ Solution	28
4.4.1	The basic idea	28
4.4.2	Why $\sqrt{L}$	29
4.4.3	Compute overhead	30
4.4.4	Code example	30
4.5	The OS Analogy: Demand Paging at Layer Granularity	31

4.6	Composition: Two Levels of Recomputation	32
5	Discussion	34
5.1	Same Ideas, Different Hardware	34
5.2	Why the Gap Is Wider on GPUs	35
5.3	Co-Evolution of Hardware and Algorithms	36
5.4	Open Questions	37
5.5	Conclusion	39
	Bibliography	42

List of Figures

2.1	Standard attention as three CUDA kernels. The $N \times N$ intermediates S and P are written to HBM by one kernel and read back by the next. Three full $\Theta(N^2)$ HBM round-trips are required per attention layer.	10
3.1	Block decomposition of $S = QK^\top/\sqrt{d}$ used by FlashAttention. The outer loop fixes a row block Q_i in shared memory. The inner loop walks the column blocks (K_j, V_j) . The current tile S_{ij} is computed inside SRAM and consumed in place; it is never written to HBM.	15
3.2	The FA3 three-way asynchronous pipeline at steady state. From T_3 onward all three hardware units are in flight on different tiles. This is what turns the 35% utilization of FA2 on H100 into the $\sim 85\%$ utilization of FA3.	19
4.1	Memory profile of the attention backward pass. The standard implementation persists the full $N \times N$ matrices S and P in HBM. FlashAttention’s backward stores only the per-row scalar L_i (N values total) and recomputes S and P on-chip from Q, K, V during the backward.	26

4.2	Gradient checkpointing on a network of $L = 9$ layers with segment size $k = 3$. Top: the standard scheme stores all activations during the forward, giving $\mathcal{O}(L)$ memory. Bottom: the checkpointed scheme stores only every k -th activation and replays the forward inside each segment during backward. With $k = \sqrt{L}$ the memory becomes $\mathcal{O}(\sqrt{L})$, at the cost of one extra forward pass.	29
-----	---	----

List of Tables

2.1	The H100 memory hierarchy. Latencies and bandwidths are typical values; per-SM aggregate bandwidth depends on the access pattern.	7
2.2	Working set of standard self-attention compared with on-chip SRAM. Numbers assume FP16 storage of $S = QK^\top$	9
3.1	Direct correspondence between the FA3 design and the classical producer/consumer setup in operating systems.	20
3.2	FlashAttention performance comparison on H100 (long-sequence attention, BF16 unless noted).	21
4.1	FlashAttention forward+backward versus standard attention on A100. Numbers from Dao et al. [6] for GPT-2 medium, sequence length 1,024, batch size 64.	27
4.2	Mapping between gradient checkpointing and demand paging. . .	31
5.1	Mapping between GPU memory-optimization techniques and classical operating systems ideas.	41

Chapter 1

Introduction

1.1 The Memory Wall in Modern Deep Learning

Transformer models [27] have grown rapidly along two axes over the past several years: more parameters and longer input sequences. Both push against the same hardware resource, GPU memory. Practitioners now routinely speak of a memory wall, a regime in which the bottleneck is not the throughput of the arithmetic units but the bandwidth and capacity of the memory hierarchy that feeds them. The phrase itself goes back to Wulf and McKee [29], who pointed out three decades ago that processor speeds were improving exponentially faster than DRAM speeds. The Roofline model of Williams, Waterman, and Patterson [28] formalized this gap as a graph: any kernel whose arithmetic intensity (FLOPs per byte of memory traffic) sits below the ridge point is bound by memory rather than by compute. Transformer attention sits well below the ridge on H100.

The growth of language models has made the gap practically urgent. Pre-trained transformer models have moved from BERT-large at 340 M parame-

ters [8], to GPT-3 at 175 B [1], to LLaMA family models at 7 to 70 B [26], with the compute-optimal scaling laws of Hoffmann et al. [11] showing that this growth is not slowing. Distributed-training systems such as Megatron-LM [25] and ZeRO [23] address the memory problem by sharding parameters across many devices, but they do not change the fundamental fact that a single device must still execute its slice efficiently.

Two facts make this bottleneck particularly sharp for the Transformer architecture. First, the self-attention operator

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V \quad (1.1)$$

has memory cost quadratic in the sequence length N , since the intermediate score matrix $S = QK^\top$ has shape $N \times N$. Doubling N quadruples the size of this intermediate. Second, accelerators such as the H100 have a very wide gap between fast on-chip memory and slow off-chip memory. The shared memory available to one streaming multiprocessor (SM) is about 228 KB, while HBM3 main memory is 80 GB. The capacity ratio is on the order of 3.5×10^5 . For any non-trivial sequence length, the working set of attention is forced out of fast memory and into HBM, where it is read and written multiple times per layer.

1.2 Connection to Operating Systems

The deep learning systems community has converged on two families of techniques to address this. From the perspective of an operating systems course, both are old OS ideas applied at a new granularity.

The first family is *tiling*. The data is decomposed into blocks small enough to fit in fast memory, and the kernel processes one block at a time. This is the cache-blocking idea that has been used in numerical linear algebra for decades, going back at least to Lam, Rothberg, and Wolf [15], with later developments such

as the cache-oblivious algorithms of Frigo et al. [9]. FlashAttention [6] applies it to the attention kernel, so that the $N \times N$ score and probability matrices never appear in HBM. An independent line of work by Rabe and Staats [22] arrived at a closely related $\mathcal{O}(N)$ -memory attention algorithm. FlashAttention-3 [24] layers a producer/consumer pipeline with a circular shared-memory buffer on top of the tile loop. This is the pattern operating systems use to overlap computation with asynchronous I/O and to keep DMA engines busy through double or triple buffering.

The second family is *recomputation*. Intermediate values that would otherwise occupy memory are dropped, and regenerated from inputs whenever they are next needed. This is the compute-for-memory trade. It corresponds to demand paging in an operating system, where a page that has been evicted from RAM is brought back through a fault when next accessed, with the cost of an extra disk read substituted for resident memory. A modern incarnation of this idea is PagedAttention [14], which manages the KV cache during inference using virtual-memory paging at the granularity of attention blocks. FlashAttention’s backward pass applies the recomputation trade at the operator level: it drops S and P and reconstructs them on-chip from Q , K , V during the backward sweep. Gradient checkpointing [2], building on the older Revolve algorithm of Griewank and Walther [10], applies it at the network level: it stores only every $\sqrt{L}$ -th layer’s activations and replays the forward pass within each segment during backward. Korthikanti et al. [13] have since shown that with careful sequence parallelism most of the recomputation overhead can be eliminated.

1.3 Organization

The remainder of the report is organized as follows. [Chapter 2](#) reviews the H100 memory hierarchy, explains why the SRAM/DRAM gap is what it is, and frames attention as a working-set problem. [Chapter 3](#) covers the tiling family, beginning with the original FlashAttention algorithm and online softmax, and ending with the FlashAttention-3 warp-specialized pipeline. [Chapter 4](#) covers the recomputation family, beginning with FlashAttention’s backward pass and ending with gradient checkpointing, and explains how the two compose in production training stacks. [Chapter 5](#) discusses what the operating-systems analogy explains, where it stops, and what open questions remain.

Chapter 2

Preliminary

This chapter establishes the hardware and problem setting that the rest of the report relies on. [Section 2.1](#) explains why the layers of a GPU’s memory hierarchy have the speeds they do. [Section 2.2](#) gives the concrete numbers for the H100. [Section 2.3](#) introduces the Tensor Memory Accelerator, a piece of H100-specific hardware that the FlashAttention-3 pipeline depends on. [Section 2.4](#) states the working-set problem of attention. [Section 2.5](#) previews the two solution strategies developed in the following chapters.

2.1 Why Different Memories Have Different Speeds

The speed differences across a GPU’s memory hierarchy are not design choices that could in principle be reversed. They come from how a bit is stored in silicon. Two storage technologies dominate inside a GPU package, and they sit at opposite ends of a density-versus-speed trade-off.

SRAM. A bit in static RAM is stored in a six-transistor bistable latch built from two cross-coupled inverters. The bit is the state of the latch. There is no

charge to refresh, and reading or writing the cell is a transistor switching event. This makes SRAM extremely fast, with single-digit nanosecond access times. The 6T cell is large in silicon area, however, so SRAM has low density and is expensive per bit. This is why on-chip SRAM is measured in kilobytes and megabytes rather than gigabytes. SRAM is used for registers, shared memory, the L1 data cache, and the L2 cache.

DRAM. A bit in dynamic RAM is stored as charge on a small capacitor, with one access transistor on top of it (the so-called 1T1C cell). Reading the cell discharges the capacitor and triggers a write-back; the capacitor also leaks charge over time and must be refreshed periodically. Charging and discharging a capacitor is intrinsically slower than switching a transistor, by roughly an order of magnitude. The compensation is in density. A 1T1C cell is small, so DRAM stores about $5\times$ more bits per unit area than SRAM. This is why off-chip main memory is measured in gigabytes, but its latency is in the hundreds of cycles. DRAM is used for HBM3 main memory on the GPU and for DDR on the host.

The implication is that a program that wants both density and speed must explicitly manage data movement between the two technologies. This is exactly what the techniques in [Chapter 3](#) and [Chapter 4](#) do.

2.2 The H100 Memory Hierarchy

[Table 2.1](#) summarizes the memory layers visible to a CUDA kernel on the H100. The numbers are drawn from the NVIDIA H100 architecture whitepaper [\[20\]](#) and the public CUDA programming guide.

A few features of this hierarchy deserve emphasis.

Table 2.1: The H100 memory hierarchy. Latencies and bandwidths are typical values; per-SM aggregate bandwidth depends on the access pattern.

Level	Type	Capacity	Bandwidth	Latency (cyc.)
Registers	SRAM	256 KB / SM	n/a	~ 0
Shared memory / L1	SRAM (6T)	228 KB / SM	~ 33 TB/s	~ 20 to 30
L2 cache	SRAM	50 MB	~ 12 TB/s	~ 200
HBM3 (VRAM)	3D DRAM	80 GB	3.35 TB/s	~ 400 to 500
CPU DRAM	DDR5	hundreds of GB	~ 64 GB/s	$\sim 10^5$

HBM3 is a hybrid. HBM is sometimes spoken of as if it were fast memory, but its cell-level latency is still that of DRAM. What HBM gives up to GDDR or DDR is not latency but bandwidth concentration. HBM achieves its bandwidth in two ways. Eight DRAM dies are stacked vertically and connected through the silicon by through-silicon vias (TSVs), packing more banks per unit footprint. Each stack also exposes a 1024-bit interface, much wider than the ~ 384 -bit interface of GDDR6. The H100 uses five active HBM3 stacks at 1024 bits each, giving a total bus width of 5,120 bits and a measured bandwidth of about 3.35 TB/s. Even so, HBM3 is roughly an order of magnitude slower than aggregate per-SM SRAM bandwidth, and its latency is more than an order of magnitude higher.

The L2 cache is not large enough to substitute for tiling. The 50 MB L2 is shared across all 132 SMs. It can absorb working sets in the tens of megabytes, but for an attention intermediate of 32 MB at $N=4096$ (the smallest setting of practical interest, see [Section 2.4](#)), the L2 can hold at most one such matrix at a time. There is no room for multiple tiles in flight, prefetched data, or other tensors. For longer contexts the L2 is hopelessly outmatched.

Off-package interconnect is much slower. The H100 communicates with CPU memory over PCIe Gen5 at 128 GB/s aggregate, and with peer GPUs over NVLink 4.0 at 900 GB/s aggregate. NVLink, the fastest GPU-to-GPU link available, is roughly $4\times$ slower than HBM3 on a single GPU. CPU DRAM is always the slowest link in any pipeline that touches it. Any technique that avoids spilling to host memory therefore wins by a wide margin.

2.3 The Tensor Memory Accelerator (TMA)

The Hopper (H100) architecture [3] introduced a new piece of hardware called the Tensor Memory Accelerator (TMA) [20]. The TMA is best understood as a DMA engine specialized for high-dimensional tensor copies between HBM and shared memory. Its key properties are as follows.

- A single thread issues a transfer by writing a copy descriptor that names the source pointer, destination pointer, dimensions, strides, and bounds.
- The TMA hardware then computes the actual addresses, handles boundary conditions, and runs the transfer asynchronously, while the issuing thread is free to do other work.
- Other warps in the SM can keep computing on already-loaded data while the TMA fetches the next tile.
- Because the TMA does the address arithmetic, the regular CUDA cores and registers that would otherwise have been used for indexing are freed up for compute.

The last point matters in practice. On A100 [4], loading a tile of K and V for an attention kernel cost not only HBM bandwidth but also a significant amount

of register pressure and arithmetic on the CUDA cores doing the indexing. On H100, the TMA absorbs that work into dedicated silicon. [Section 3.4](#) shows how FlashAttention-3 builds an entire producer/consumer pipeline around this fact.

2.4 Self-Attention and Its Working Set

The standard self-attention operator from (1.1) is computed as

$$S = \frac{QK^\top}{\sqrt{d}}, \quad P = \text{softmax}(S), \quad O = PV, \quad (2.1)$$

with $Q, K, V \in \mathbb{R}^{N \times d}$ and $S, P \in \mathbb{R}^{N \times N}$. The intermediate matrices S and P each contain N^2 scalars. [Table 2.2](#) gives the size of S in FP16 for several representative sequence lengths and compares it to the per-SM SRAM capacity.

Table 2.2: Working set of standard self-attention compared with on-chip SRAM. Numbers assume FP16 storage of $S = QK^\top$.

Sequence length N	Size of S (FP16)	H100 SRAM/SM	Ratio
4,096 (typical training)	32 MB	228 KB	140×
32,768 (long context)	2 GB	228 KB	9,000×
131,072 (GPT-4-class)	32 GB	228 KB	140,000×

Even the smallest setting in the table puts S two orders of magnitude above per-SM SRAM. At GPT-4-class context lengths S approaches the size of the entire HBM, and that is before multiplying by the batch size and the number of attention heads. So S and P cannot live in fast memory and must instead be staged through HBM.

A naive CUDA implementation realizes (1.1) as three separate kernels:

Kernel 1. Compute $S = QK^\top$, write S ($N \times N$) to HBM.

Kernel 2. Read S from HBM, compute $P = \text{softmax}(S)$, write P to HBM.

Kernel 3. Read P from HBM, compute $O = PV$, write O to HBM.

The three-kernel structure is forced by the CUDA programming model. Between kernel launches, registers and shared memory are not preserved, so the only way for one kernel to communicate with the next is through HBM. Figure 2.1 sketches the resulting data flow.

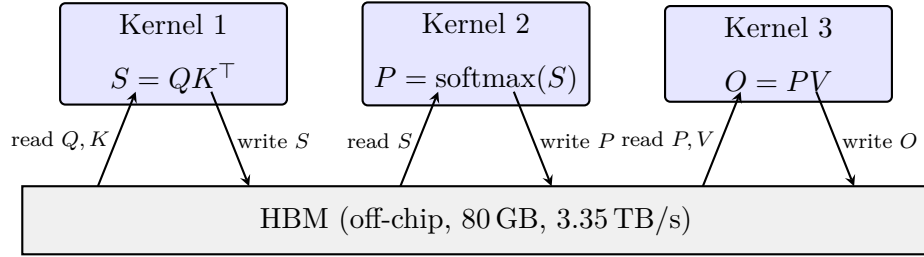


Figure 2.1: Standard attention as three CUDA kernels. The $N \times N$ intermediates S and P are written to HBM by one kernel and read back by the next. Three full $\Theta(N^2)$ HBM round-trips are required per attention layer.

The compulsory HBM traffic per attention layer is therefore at least

$$\begin{aligned}
 & 2 \cdot N \cdot d \quad (\text{write/read of } Q, K, V, O) \\
 & + 4 \cdot N^2 \quad (\text{write of } S, \text{ read of } S, \text{ write of } P, \text{ read of } P). \quad (2.2)
 \end{aligned}$$

For $N=4096$ and $d=128$, the intermediate traffic is $4 \cdot 32 \text{ MB} = 128 \text{ MB}$, and dominates the 4 MB of input/output traffic. The arithmetic intensity (FLOPs per byte) is correspondingly low, and the H100 tensor cores [16] stall waiting for HBM. Attention in this naive form is memory-bound, not compute-bound.

2.5 Two Solution Strategies

There are two natural ways to attack the working-set problem above, and they occupy the next two chapters of the report.

Tiling (Chapter 3). If the data does not fit in fast memory, break it into blocks that do, and compute one block at a time. By sizing blocks of Q , K , and V so that each block fits comfortably in shared memory, the intermediates S and P can be computed, used, and discarded entirely on-chip. The three separate kernels of the baseline collapse into a single fused kernel that walks the tile loop. This is the approach of FlashAttention [6] and FlashAttention-3 [24].

Recomputation (Chapter 4). If a value is too expensive to store, drop it and recompute it when next needed. Recomputation is the compute-for-memory trade. It applies when even tiling cannot bring the working set within the available memory, which is typically the case during training, where the activations of every layer must persist for the backward pass. FlashAttention’s backward applies recomputation at the operator level by dropping S and P and reconstructing them on-chip during backward. Gradient checkpointing [2] applies it at the network level by dropping most layer activations and replaying the forward pass during backward.

Both strategies are direct re-applications of operating systems memory-management ideas to a new hardware regime, a point developed in Chapter 5.

Chapter 3

Tiling: FlashAttention to FlashAttention-3

This chapter develops the first of the two strategies introduced in [Section 2.5](#). [Section 3.1](#) explains what tiling buys at two distinct time scales. [Section 3.2](#) reviews the three-kernel baseline. [Section 3.3](#) presents FlashAttention [6], including the online softmax that makes a streaming computation possible. [Section 3.4](#) presents FlashAttention-3 [24] and its warp-specialized producer/consumer pipeline. [Section 3.5](#) reports the measured performance numbers. [Section 3.6](#) sets out the explicit OS analogy.

3.1 Two Layers of Benefit

Tiling buys two separate kinds of benefit on different time scales.

Spatial benefit (FA1, FA2). When the blocks are sized so that the tile working set, a slice of Q , a slice of K , a slice of V , plus the local S and P , fits inside per-SM shared memory, the score and probability matrices never have to be writ-

ten to HBM. The three separate kernels of the baseline can be fused into one. This eliminates the dominant $\Theta(N^2)$ HBM round-trips and reduces HBM I/O to $\Theta(N^2 d^2 / M)$, where M is the size of fast memory [6]. On A100, FlashAttention v2 reaches about 70% of peak BF16 tensor-core utilization on long-sequence attention [5].

Temporal benefit (FA3). The tile loop introduced by tiling itself creates an opportunity for pipeline parallelism. Each iteration of the loop loads a tile of K, V from HBM, computes a matmul, and applies softmax to the result. The first uses HBM bandwidth, the second uses the tensor cores, and the third uses the special-function unit. On H100 these three resources can run concurrently if the schedule is right. The load for tile $i + 1$, the matmul for tile i , and the softmax for tile $i - 1$ can all be in flight at the same time. Without the loop there would be no schedule to exploit. FlashAttention-3 turns this idea into roughly a $2.4\times$ speedup over FA2 on H100 [24].

The two layers are independent. The spatial benefit was visible already on A100. The temporal benefit only matters when the hardware imbalance between matmul throughput and softmax throughput is large enough that running them serially leaves the matmul unit idle. This is the situation on H100, where the BF16 matmul rate of 989 TFLOPS is roughly $256\times$ the special-function-unit rate of about 3.9 TFLOPS for the exponentials inside softmax [24].

3.2 Standard Attention: Three Separate Kernels

The baseline of Section 2.4 is restated here for direct comparison with FlashAttention. The naive implementation of (1.1) is:

Kernel 1. Compute $S = QK^\top/\sqrt{d}$, write $S \in \mathbb{R}^{N \times N}$ to HBM.

Kernel 2. Read S from HBM, compute $P = \text{softmax}(S)$ row-wise, write P to HBM.

Kernel 3. Read P from HBM, compute $O = PV$, write $O \in \mathbb{R}^{N \times d}$ to HBM.

Each kernel writes its output to HBM and the next kernel reads it back, as illustrated in [Figure 2.1](#). The arithmetic intensity is correspondingly low, and the H100 tensor cores stall.

3.3 FlashAttention: Tiling and Online Softmax

3.3.1 Block decomposition of S

FlashAttention [6] restructures the computation as a single fused kernel that walks an outer loop over blocks of Q and an inner loop over blocks of K, V . [Figure 3.1](#) sketches the block decomposition.

3.3.2 The forward algorithm

The simplified forward pass following the FA2 formulation [5] is given as [Algorithm 1](#).

The intermediates S_{ij} , $\tilde{P}_{ij}$ and the running statistics $m_i, \ell_i, \tilde{O}_i$ all live in shared memory and registers. The only HBM traffic is the input/output streaming of Q, K, V, O and the per-row log-sum-exp L_i , which is needed by the backward pass ([Chapter 4](#)). Total HBM I/O drops from $\Theta(N^2)$ in the baseline to $\Theta(N^2 d^2/M)$ in standard cache-blocking analysis, where M is the per-SM SRAM budget [6].

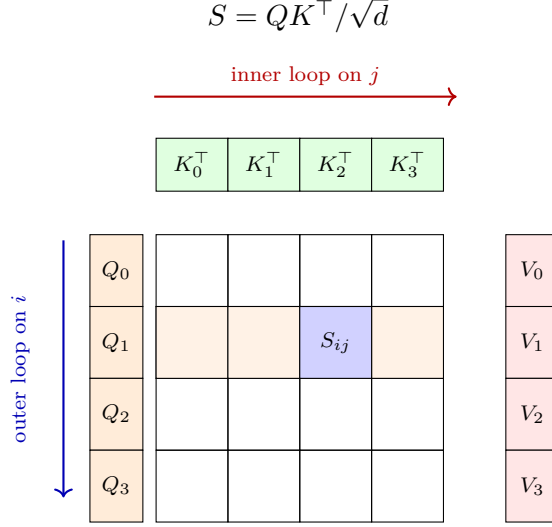


Figure 3.1: Block decomposition of $S = QK^\top / \sqrt{d}$ used by FlashAttention. The outer loop fixes a row block Q_i in shared memory. The inner loop walks the column blocks (K_j, V_j) . The current tile S_{ij} is computed inside SRAM and consumed in place; it is never written to HBM.

3.3.3 Online softmax

The naive worry about [Algorithm 1](#) is correctness. The softmax of a row x requires the global maximum,

$$\text{softmax}(x)_k = \frac{\exp(x_k - \max_j x_j)}{\sum_l \exp(x_l - \max_j x_j)}, \quad (3.1)$$

and a streaming algorithm that processes blocks of x in turn does not, at any intermediate step, know what the eventual global maximum will be. The fix is the online softmax of Milakov and Gimelshein [19]. The algorithm maintains a running maximum and a running denominator, and rescales them when a new block reveals a larger maximum.

Suppose the row of scores has been split into blocks B_1, B_2 , with local maxima

Algorithm 1 FlashAttention (FA2) forward, simplified.

Require: $Q, K, V \in \mathbb{R}^{N \times d}$; block sizes B_r, B_c .

Ensure: $O \in \mathbb{R}^{N \times d}$ and per-row log-sum-exp $L \in \mathbb{R}^N$.

```

1: for each  $Q$ -block  $Q_i$  of size  $B_r \times d$  do
2:   Load  $Q_i$  into SRAM
3:   Initialize  $m_i \leftarrow -\infty, \ell_i \leftarrow 0, \tilde{O}_i \leftarrow 0$ 
4:   for each  $(K, V)$ -block  $(K_j, V_j)$  of size  $B_c \times d$  do
5:     Load  $K_j, V_j$  into SRAM
6:      $S_{ij} \leftarrow Q_i K_j^\top / \sqrt{d}$  ▷ computed and held in SRAM
7:      $m^{\text{new}} \leftarrow \max(m_i, \text{rowmax}(S_{ij}))$ 
8:      $\tilde{P}_{ij} \leftarrow \exp(S_{ij} - m^{\text{new}})$ 
9:      $\ell_i \leftarrow e^{m_i - m^{\text{new}}} \cdot \ell_i + \text{rowsum}(\tilde{P}_{ij})$ 
10:     $\tilde{O}_i \leftarrow e^{m_i - m^{\text{new}}} \cdot \tilde{O}_i + \tilde{P}_{ij} V_j$ 
11:     $m_i \leftarrow m^{\text{new}}$ 
12:  end for
13:   $O_i \leftarrow \tilde{O}_i / \ell_i$  ▷ single division at end of  $Q$ -block
14:   $L_i \leftarrow m_i + \log(\ell_i)$ 
15:  Write  $O_i, L_i$  to HBM
16: end for

```

m_1, m_2 and global maximum $m = \max(m_1, m_2)$. Define

$$\ell^{(1)} = \sum_{x \in B_1} e^{x - m_1}, \quad \ell^{(2)} = \sum_{x \in B_2} e^{x - m_2}. \quad (3.2)$$

After processing block 1, the algorithm holds the local pair $(m_1, \ell^{(1)})$. When block 2 arrives, the new pair (m, ℓ) is

$$m = \max(m_1, m_2), \quad \ell = e^{m_1 - m} \cdot \ell^{(1)} + e^{m_2 - m} \cdot \ell^{(2)}. \quad (3.3)$$

The previously accumulated denominator is corrected by the multiplicative factor

$e^{m_{\text{old}} - m_{\text{new}}}$, which compensates for having normalized against the wrong (smaller) maximum. The same correction is applied to the running output $\tilde{O}_i$ in line 10 of [Algorithm 1](#).

3.3.4 Numerical example

Consider scores $x = [1, 3, 2, 4]$ split into two blocks of two. After block 1 ($x = [1, 3]$), the local maximum is $m_1 = 3$ and the local denominator is $\ell^{(1)} = e^{1-3} + e^{3-3} = e^{-2} + 1 \approx 1.135$. After block 2 ($x = [2, 4]$), the local maximum is $m_2 = 4$ and the local denominator is $\ell^{(2)} = e^{2-4} + e^{4-4} \approx 1.135$. The combined denominator from (3.3) is

$$\ell = e^{3-4} \cdot 1.135 + e^{4-4} \cdot 1.135 \approx 0.368 \cdot 1.135 + 1.135 \approx 1.553. \quad (3.4)$$

For the first element, $\text{softmax}(x)_1 = e^{1-4}/1.553 \approx 0.0498/1.553 \approx 0.032$, which agrees with the result of computing softmax in one shot.

3.3.5 Complexity and intermediate result on A100

The forward of FlashAttention satisfies the following.

- Auxiliary HBM memory drops from $\Theta(N^2)$ to $\Theta(N)$. The only persistent per-row state stored in HBM is the log-sum-exp $L_i = m_i + \log \ell_i$.
- HBM I/O drops from $\Theta(N^2)$ to $\Theta(N^2 d^2 / M)$, an asymptotic improvement when M is non-trivial.
- Floating-point operations stay at $\Theta(N^2 d)$, the same as the baseline.

In practice, FA2 reaches about 70% of peak BF16 tensor-core utilization on A100. On H100, the same algorithm reaches only about 35% [24]. The reason is not that the algorithm became wrong, but that H100 grew its matmul throughput

much faster than its softmax throughput, leaving the special-function unit, not HBM, as the new bottleneck. Closing that gap is the job of FlashAttention-3.

3.4 FlashAttention-3: Three-Way Asynchronous Pipeline

3.4.1 The throughput gap

The numerical gap that motivates FlashAttention-3 is captured by two ratios on H100. The BF16 matmul throughput on the tensor cores is about 989 TFLOPS, while the special-function-unit throughput for the exponentials inside softmax is about 3.9 TFLOPS, a roughly $256\times$ asymmetry [24]. If matmul and softmax run serially within the same tile, then for every microsecond the SFU is busy with $\exp(\cdot)$, the matmul unit is idle. This is why FA2 saturates at about 35% on H100.

3.4.2 Warp specialization and the producer/consumer pattern

FlashAttention-3 splits the warps within an SM into specialized roles, and pipelines their work through a circular shared-memory buffer.

- **Producer warps.** Issue TMA loads of the next (K, V) tile from HBM into a slot in a circular shared-memory buffer. Producer warps need very few registers, since the TMA does the address arithmetic in dedicated hardware. They are configured to donate registers to the consumer warps via the `setmaxnreg` instruction, which lets the consumers run with larger GEMM tile sizes and higher arithmetic intensity.

- **Consumer warps (matmul).** Run the WGMMA (warp-group matrix multiply-accumulate) on the current (K, V) tile against the resident Q tile, producing the local score matrix S_{ij} .
- **Consumer warps (softmax).** Apply the special-function-unit operations (max, exp, rescale) to the previous tile's S_{ij} , while the matmul warps work on the current tile.

At steady state, three operations are simultaneously in flight on different hardware units:

$$\text{Load}(i+1) \quad \parallel \quad \text{MatMul}(i) \quad \parallel \quad \text{Softmax}(i-1). \quad (3.5)$$

Figure 3.2 shows the timeline. The producer/consumer pattern is the standard operating systems primitive for overlapping I/O with computation. The TMA plays the role of the producer, the matmul and softmax warps play the role of consumers, and the circular shared-memory buffer plays the role of the bounded queue. Synchronization is through hardware semaphores (the `mbarrier` objects on H100).

Time	T_1	T_2	T_3	T_4	T_5
Producer (TMA)	Load K_1V_1	Load K_2V_2	Load K_3V_3	Load K_4V_4	idle
Consumer (WGMMA)	idle	$QK_1^\top$	$P_1V_1, QK_2^\top$	$P_2V_2, QK_3^\top$	$P_3V_3, QK_4^\top$
Consumer (softmax)	idle	idle	softmax ₁	softmax ₂	softmax ₃

Figure 3.2: The FA3 three-way asynchronous pipeline at steady state. From T_3 onward all three hardware units are in flight on different tiles. This is what turns the 35% utilization of FA2 on H100 into the $\sim 85\%$ utilization of FA3.

3.4.3 The OS pattern, made concrete

The FA3 design mirrors the classical producer/consumer setup quite closely. In an operating system, the producer and consumer are typically threads or processes, the queue is a shared in-memory buffer (often a ring buffer for fixed-size items), and synchronization uses semaphores or condition variables. In FA3, the producer warps and consumer warps are warp-groups within an SM, the queue is a circular shared-memory buffer with a small number of slots (commonly 2 or 3, giving double or triple buffering), and synchronization uses hardware `mbarrier` semaphores. The mapping is summarized in [Table 3.1](#).

Table 3.1: Direct correspondence between the FA3 design and the classical producer/consumer setup in operating systems.

Operating systems primitive	FA3 implementation
Producer thread or process	Producer warp issuing TMA loads
Consumer thread or process	Consumer warps running WGMMA / softmax
Bounded queue	Circular shared-memory buffer (2 to 3 slots)
Semaphore / condition variable	Hardware <code>mbarrier</code>
DMA engine	TMA
Double / triple buffering	2-slot / 3-slot circular SMEM buffer

3.5 Performance Comparison

[Table 3.2](#) summarizes the reported numbers on H100 [24]. FA3 in BF16 reaches roughly 85% of peak BF16 tensor-core throughput, 2.4 $\times$ faster than FA2 on the same hardware. Adding FP8 [18] brings another $\sim 1.5\times$ on top.

Table 3.2: FlashAttention performance comparison on H100 (long-sequence attention, BF16 unless noted).

Implementation	H100 TFLOPS	Utilization	Speedup vs FA2
FlashAttention-2 (baseline)	~ 350	$\sim 35\%$	$1.0\times$
FlashAttention-3 (BF16)	~ 840	$\sim 85\%$	$2.4\times$
FlashAttention-3 (FP8)	$\sim 1,300$	n/a	$3.7\times$

The interpretation is straightforward. The jump from FA2 to FA3 in BF16 is the temporal-pipeline benefit of [Section 3.1](#): the matmul unit, idle on FA2 while softmax ran, is now busy on a different tile while softmax runs. The further jump to FP8 is the orthogonal benefit of using a smaller numerical format, which halves the bytes per value and doubles the throughput on the FP8 tensor cores, combined with FA3’s careful handling of the per-block rescaling that low-precision attention requires.

3.6 The OS Analogy in One Picture

The two layers of benefit identified in [Section 3.1](#) correspond to two distinct OS ideas. The spatial benefit of FlashAttention is the cache-blocking pattern that has been used in numerical linear algebra and OS-aware programming for decades. The temporal benefit of FlashAttention-3 is the producer/consumer pattern with a bounded buffer, the canonical example used in operating systems textbooks to introduce semaphores and asynchronous I/O.

What is genuinely new is not the patterns themselves but the granularity at which they are applied. Cache blocking on a CPU is typically a software-only technique on a hierarchy with $\sim 1,000\times$ capacity ratio between L1 and

DRAM. On the H100 the same pattern is applied across a $\sim 350,000\times$ ratio, with hardware support (TMA, WGMMMA, `mbarrier`) explicitly designed to make the pattern fast. [Section 5.3](#) returns to this point.

Chapter 4

Recomputation: FlashAttention Backward and Gradient Checkpointing

The previous chapter showed that tiling reduces the auxiliary HBM memory of attention’s forward from $\Theta(N^2)$ to $\Theta(N)$. This is enough to make inference and the forward pass of training tractable. But training also has a backward pass, and in the standard formulation the backward needs every layer’s intermediate activations from the forward. As [Section 4.3](#) shows, those activations are typically the dominant memory consumer in a training step, often eclipsing the model weights by an order of magnitude.

This chapter develops the second strategy of [Section 2.5](#), which is recomputation. The principle is straightforward. Drop intermediates, regenerate them from inputs when needed, and accept some additional compute as the cost. [Section 4.2](#) applies the principle at the level of a single attention operator, using FlashAttention’s backward as the case study. [Section 4.3](#) explains why activations dominate training memory. [Section 4.4](#) applies the principle at the level of an entire net-

work, using gradient checkpointing [2] as the case study. [Section 4.5](#) draws the analogy with demand paging. [Section 4.6](#) discusses how the operator-level and network-level techniques compose in practice.

4.1 The Compute-for-Memory Trade-off

Every recomputation technique in this chapter rests on the following observation. If a quantity X is expensive to store (it is large, or it lives in a level of the hierarchy that is full) and cheap to regenerate from already-stored inputs Y , then we are better off discarding X and recomputing it as $X = f(Y)$ when next needed. The cost of this trade is some extra arithmetic; the benefit is reduced memory pressure. Because GPU workloads are typically memory-bound rather than compute-bound, the extra arithmetic is often essentially free in wall-clock terms.

This is the bargain that demand paging strikes in an operating system. A page that has been evicted to swap is not gone. It can be brought back from disk when next accessed, paying an extra disk read in exchange for a reduced resident set. Copy-on-write and zero-fill pages take the same idea further: the kernel actively recomputes (or trivially generates) page contents rather than storing them eagerly. The two GPU techniques in this chapter are the same trade in a different setting.

4.2 FlashAttention Backward as Recomputation

4.2.1 The standard backward

In a standard attention implementation, the forward writes S and P (each $N \times N$) to HBM, and the backward reads them back to compute the gradients

$$\frac{\partial \mathcal{L}}{\partial Q}, \quad \frac{\partial \mathcal{L}}{\partial K}, \quad \frac{\partial \mathcal{L}}{\partial V}. \quad (4.1)$$

The HBM traffic of the backward is therefore comparable to that of a naive forward, plus the upstream gradient $\partial \mathcal{L} / \partial O$. The persistent memory required by attention alone is $\Theta(N^2)$ per layer, which is exactly the cost [Chapter 3](#) removed from the forward.

4.2.2 FlashAttention’s backward

FlashAttention recovers a small memory profile by recomputing S and P on-chip during the backward. The forward pass stores only the per-row log-sum-exp $L_i = m_i + \log \ell_i$ of total size N . The backward then walks the same tile loop as the forward, and for each tile (Q_i, K_j, V_j) does the following.

1. Recompute $S_{ij} = Q_i K_j^\top / \sqrt{d}$ on-chip with one tile-sized matmul.
2. Recover $P_{ij} = \exp(S_{ij} - L_i)$ with a single elementwise operation.
3. Use S_{ij} , P_{ij} and the upstream gradient $\partial \mathcal{L} / \partial O$ to compute the contributions to $\partial \mathcal{L} / \partial Q$, $\partial \mathcal{L} / \partial K$, $\partial \mathcal{L} / \partial V$, all in SRAM, with no HBM reads or writes of the $N \times N$ matrices.

Step 2 deserves comment. The stored L_i already encodes the exact global normalization of row i , since it is the result that the online softmax of [Section 3.3](#) converged to during the forward. The backward therefore does not have to redo

the streaming online-softmax bookkeeping. It needs only one elementwise exponential per entry. So the recomputation is cheap: a single matmul plus an elementwise op replaces what would otherwise have been a full $N \times N$ HBM read.

Figure 4.1 contrasts the memory profile of the standard backward with that of FlashAttention.

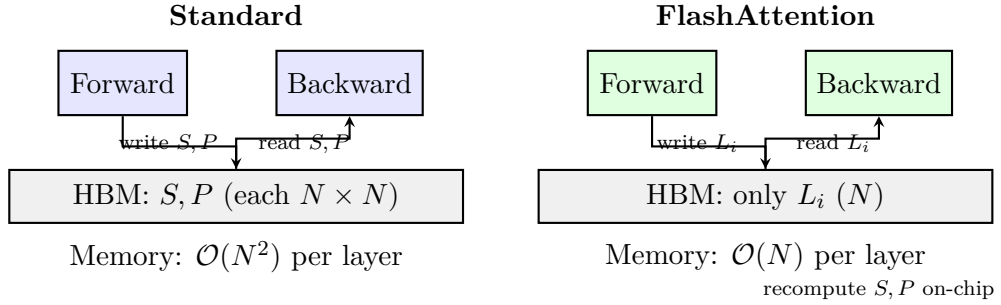


Figure 4.1: Memory profile of the attention backward pass. The standard implementation persists the full $N \times N$ matrices S and P in HBM. FlashAttention’s backward stores only the per-row scalar L_i (N values total) and recomputes S and P on-chip from Q, K, V during the backward.

4.2.3 Empirical numbers

Dao et al. [6] report concrete numbers for GPT-2 medium on A100, sequence length 1024, batch size 64, summarized in Table 4.1.

FlashAttention does about 13% more arithmetic than the standard implementation because of the recomputation work. In return it does $9.2\times$ less HBM traffic and runs $5.7\times$ faster in wall-clock time. When a workload is memory-bound, trading a small amount of extra compute for a large reduction in memory traffic is a winning bargain.

Table 4.1: FlashAttention forward+backward versus standard attention on A100. Numbers from Dao et al. [6] for GPT-2 medium, sequence length 1,024, batch size 64.

	Compute	HBM I/O	Wall time
Standard attention	66.6 GFLOPs	40.3 GB	41.7 ms
FlashAttention	75.2 GFLOPs	4.4 GB	7.3 ms
Ratio	$\times 1.13$	$/9.2$	$/5.7$

4.3 Why Activations Dominate Training Memory

Before proceeding to gradient checkpointing it is worth quantifying the problem it solves. People often think of a model’s memory footprint as the size of its weights. For training, this picture is misleading.

Weights are fixed-size. A model’s parameters are determined at construction time and shared across all inputs. A 7B-parameter model in FP16 [17] occupies a fixed $7 \times 10^9 \cdot 2\text{B} = 14\text{GB}$ regardless of the batch size, the sequence length, or the number of training steps. For inference this is roughly the whole story.

Activations grow with depth, batch, and sequence. Activations are the per-sample, per-layer intermediate tensors that the backward pass needs to compute gradients. For a transformer layer with hidden dimension h , batch size B , and sequence length N , the dominant activations stored per layer are on the order of $B \cdot N \cdot h$ tensors of various FP16 sizes (attention inputs, post-softmax probabilities, MLP intermediates, layer-norm statistics). For a 7B-parameter model with

32 layers, hidden dimension 4096, batch size 8, sequence length 2048, a careful accounting of training activations gives roughly 1.3 GB per layer [2], or about 40 GB across all 32 layers. Korthikanti et al. [13] give a detailed accounting of where these bytes go and show how to cut them with sequence parallelism.

The contrast. For this 7B / batch-8 / seq-2048 setting, activations (~ 40 GB) are about three times the size of the weights (14 GB). They grow linearly with batch size, sequence length, and model depth. With long-context training, activations can exceed the weights by an order of magnitude. This is the reason a model can fit in 24 GB while its training does not: the weights might fit, but the activations needed for the backward push the total well past 80 GB.

4.4 Gradient Checkpointing: The $\sqrt{L}$ Solution

Gradient checkpointing, also known as activation checkpointing, is the technique of Chen et al. [2] for reducing the activation footprint of training. It is now standard, available out of the box in PyTorch, JAX, and other frameworks.

4.4.1 The basic idea

In a network of L layers, the forward pass naively stores the activations $a_1, a_2, \dots, a_L$ for use during the backward. Gradient checkpointing instead stores only a small subset of these activations (the *checkpoints*) and reconstructs the rest by replaying the forward pass during the backward.

Concretely, with segment size k , every k -th activation is stored: $a_1, a_{k+1}, a_{2k+1}, \dots$. During the backward pass, when the algorithm reaches a segment that begins at a_{jk+1} and ends at $a_{(j+1)k}$, the forward of that segment is run again starting from the stored checkpoint a_{jk+1} to regenerate the intermediate activations. The gra-

dient is computed for that segment, the regenerated activations are dropped, and the algorithm proceeds to the previous segment. Figure 4.2 sketches the result.

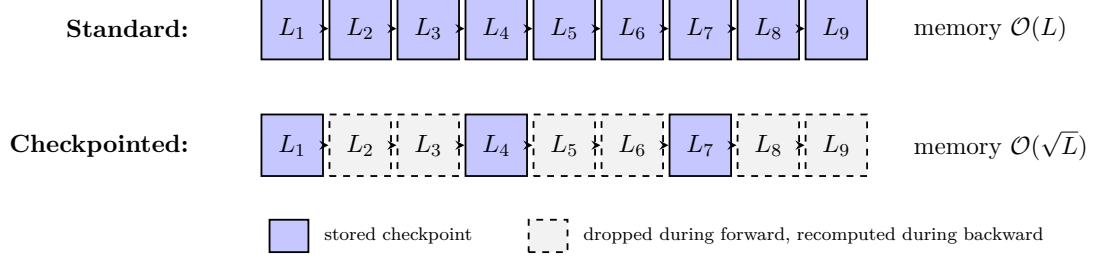


Figure 4.2: Gradient checkpointing on a network of $L = 9$ layers with segment size $k = 3$. Top: the standard scheme stores all activations during the forward, giving $\mathcal{O}(L)$ memory. Bottom: the checkpointed scheme stores only every k -th activation and replays the forward inside each segment during backward. With $k = \sqrt{L}$ the memory becomes $\mathcal{O}(\sqrt{L})$, at the cost of one extra forward pass.

4.4.2 Why $\sqrt{L}$

Two memory costs determine the total. First, the checkpoints themselves: L/k of them, each of size proportional to one layer’s activation. Second, the activations transiently held within a segment during recomputation: at most k of them at any time. The total memory at any moment is therefore proportional to

$$\frac{L}{k} + k. \quad (4.2)$$

Minimizing over k gives $k = \sqrt{L}$, at which point the memory is $2\sqrt{L}$, a sub-linear function of L rather than the original L . The asymptotic memory of training therefore drops from $\Theta(L)$ to $\Theta(\sqrt{L})$, with one extra forward pass of compute as the price.

4.4.3 Compute overhead

The compute overhead is roughly one extra forward pass. With the $\sqrt{L}$ checkpoint spacing, every segment has its forward replayed exactly once during the backward. Summing over all segments gives one full extra traversal of the network’s forward computation. A standard training step performs one forward and roughly two backwards’ worth of work [2], so an extra forward is approximately a 33% overhead, in line with the rule of thumb that practitioners report.

4.4.4 Code example

In modern frameworks the technique requires only a single line. [Listing 4.1](#) shows the PyTorch idiom.

```
1 from torch.utils.checkpoint import checkpoint
2
3 class TransformerBlock(nn.Module):
4     def forward(self, x):
5         # Replace the direct call:
6         #     x = self.attn(x)
7         # with a checkpointed call. The activations inside
8         self.attn
9         # are dropped and recomputed during backward.
10        x = checkpoint(self.attn, x, use_reentrant=False)
11        x = checkpoint(self.mlp, x, use_reentrant=False)
12        return x
```

Listing 4.1: Gradient checkpointing in PyTorch.

The wrapper hides all the bookkeeping (saving the inputs, registering a custom backward that re-runs the forward, integrating with autograd) behind a single

function call. From the user’s perspective, gradient checkpointing is a one-line change that converts $\Theta(L)$ activation memory into $\Theta(\sqrt{L})$ at the cost of one extra forward pass.

4.5 The OS Analogy: Demand Paging at Layer Granularity

The structural correspondence between gradient checkpointing and demand paging is close enough to be worth stating directly. [Table 4.2](#) gives the mapping.

Table 4.2: Mapping between gradient checkpointing and demand paging.

Concept	Demand paging	Gradient checkpointing
Resident set	Pages currently in physical RAM	Layer activations currently in HBM
Eviction	Evicted by page-replacement on pressure	Not stored at the time of forward
Fault	Page fault, serviced by reading from swap	Backward request serviced by replaying forward from nearest checkpoint
Tuning knob	Working-set size	Segment size k , optimal $k = \sqrt{L}$

The optimal $k = \sqrt{L}$ result is the gradient-checkpointing analogue of the working-set theorem: there is a sweet spot where the marginal cost of one more

resident item equals the marginal cost of one more fault. The exact algebra is different, but the shape of the trade is the same.

In both settings, the technique is invisible to the algorithm’s correctness. Demand paging does not change what a program computes; it only changes when memory is materialized. Gradient checkpointing does not change the gradients computed by autograd; it only changes when the activations are materialized. Both are transparent virtualization layers over a finite physical resource.

4.6 Composition: Two Levels of Recomputation

The operator-level recomputation of [Section 4.2](#) and the network-level recomputation of [Section 4.4](#) compose, and in production training stacks they are nearly always used together.

- Inside each transformer block, FlashAttention is used as the attention kernel. Its forward already drops S and P , and its backward recomputes them on-chip from Q, K, V and the saved L_i . This eliminates the $\Theta(N^2)$ per-layer cost of attention’s intermediates.
- Across the entire network, gradient checkpointing wraps every $\sqrt{L}$ -th layer boundary as a checkpoint, dropping the per-layer activation tensors of the uncheckpointed layers. This eliminates the $\Theta(L)$ scaling of layer-level activation storage.

The two reductions stack. The per-layer activation footprint is small (FlashAttention has shrunk it), and the layer dimension is also small (checkpointing has shrunk it). The product of the two factors is what makes long-context training of modern LLMs feasible on commodity hardware. A training step that would, in

the naive implementation, require terabytes of activation memory ends up fitting on a single 80 GB H100.

Chapter 5

Discussion

This chapter steps back from the technical details of [Chapter 3](#) and [Chapter 4](#) and asks what these techniques teach us at the level of principle. [Section 5.1](#) consolidates the OS analogies into a single table. [Section 5.2](#) discusses why the gap between fast and slow memory is so much wider on GPUs than on CPUs. [Section 5.3](#) examines the co-evolution of hardware and algorithms in this space, with FA1 to FA3 as the case study. [Section 5.4](#) identifies open research questions. [Section 5.5](#) concludes.

5.1 Same Ideas, Different Hardware

[Table 5.1](#) consolidates the analogies developed throughout the report. None of the GPU-side techniques is a new idea at the level of principle. What is new is the granularity at which the idea is applied and the scale of the gap it bridges.

The point of the table is not that FlashAttention or gradient checkpointing are merely re-statements of textbook OS material. Translating an OS pattern to a new hardware regime is non-trivial. The mapping has to identify the right hardware units to play the role of producer and consumer (the TMA and the

WGMMA tensor cores in FA3), choose the right granularity for the queue (the SMEM circular buffer), and build the right synchronization primitive (`mbarrier`). What the table does claim is that, viewed from a sufficient distance, modern GPU memory optimization is a continuation of the same memory-management story that operating systems have been telling for half a century.

5.2 Why the Gap Is Wider on GPUs

The H100 represents a regime in which the cost of explicit memory management is no longer optional. To see how unusual this is by historical standards, the analogous numbers on a CPU are useful as a comparison.

CPU L1 to DRAM ratio. A modern CPU has L1 caches of 32 to 64 KB per core and main memory of order 10^{10} bytes, a capacity ratio in the neighborhood of $10^5:1$. The bandwidth ratio between L1 and DRAM is roughly $10^2:1$, and the latency ratio is roughly 50:1 (a few cycles for L1, ~ 200 cycles for DRAM). Programs that ignore the cache hierarchy still run; they merely run slower than they could. Cache blocking is a useful optimization, not a survival requirement.

H100 SRAM to HBM ratio. The per-SM SRAM is 228 KB and HBM is 80 GB, a capacity ratio of about $3.5 \times 10^5:1$. The latency ratio is more extreme as well: ~ 30 cycles for shared memory versus ~ 500 cycles for HBM, roughly 20:1. Per byte of data delivered, HBM is much further behind because the bandwidth gap of $\sim 10:1$ stacks with the longer latency. An algorithm that does not manage the hierarchy does not merely run slowly on H100; it can leave more than 90% of the silicon idle.

The implication. Algorithms that ran adequately on CPUs without explicit memory management collapse on GPUs. This is why the entire ecosystem of GPU-aware kernels (cuBLAS, cuDNN, FlashAttention, and a long tail of custom kernels) is needed in the first place. It is also why a course on operating systems is the right intellectual preparation for a course on deep learning systems: the hard work in both cases is the same hard work of managing a finite, fast, on-chip resource against a much larger, much slower, off-chip one.

5.3 Co-Evolution of Hardware and Algorithms

The history of FlashAttention illustrates how hardware and algorithms evolve together.

FA1 was designed for A100. The original FlashAttention [6] targeted A100, where the imbalance between matmul throughput and softmax throughput was modest enough that running them serially within a tile left the matmul unit only mildly idle. With careful tiling, FA2 [5] reached about 70% of peak BF16 utilization on A100.

The same algorithm regressed on H100. When H100 arrived, the same FA2 algorithm dropped to about 35% utilization. The reason was not that the algorithm got worse. H100’s matmul throughput grew faster than its softmax throughput: BF16 tensor-core throughput rose from ~ 312 TFLOPS on A100 to ~ 989 TFLOPS on H100, while the special-function-unit throughput for $\exp(\cdot)$ scaled much more modestly. The hardware imbalance that FA2 had not had to worry about on A100 became the dominant bottleneck on H100.

FA3 was a software response to specific H100 features. FlashAttention-3 [24] closes the gap by exploiting H100-specific hardware that did not exist on A100: the TMA, the WGMMMA warp-group matmul instruction, the asynchronous `wgmma.async` variants, and the `setmaxnreg` register-reallocation instruction. None of these were available on A100. All of them were designed by the H100 architects in anticipation of producer/consumer kernels of exactly the form FA3 ended up using.

The hardware was designed for the algorithmic pattern. The TMA in particular is hard to justify if the only kernels that use it are simple ones; its complexity (the descriptor format, the tile-aware addressing, the asynchronous semantics) only pays off when one is building a producer/consumer pipeline. The H100 designers were betting on producer/consumer kernels well before FA3 was published. FA3 is in this sense the realization of a design intent already baked into the hardware.

The systems-architecture lesson from this episode is that algorithms should be written with enough abstraction to be re-targetable. FA’s tile-loop structure was the right abstraction, even as the schedule of operations within the loop had to be redesigned for H100.

5.4 Open Questions

Several questions remain open.

Non-uniform checkpoint placement. The $\sqrt{L}$ analysis of Section 4.4 assumes that all layers cost the same to recompute and to store. In modern transformers, attention-dominated layers and MLP-dominated layers have noticeably different cost profiles, and long-context attention layers can cost many times

what a short-context layer costs. A more sophisticated analysis would solve a non-uniform optimization problem to place checkpoints; some recent work goes in this direction [10], but a fully automatic, workload-aware solution that integrates with PyTorch is still an open practical problem.

Portability of the FA3 schedule. The producer/consumer pipeline of FA3 leans heavily on H100-specific primitives (TMA, WGMMA, `mbarrier`). Porting it to AMD’s CDNA3 architecture, to TPUs, or to future NVIDIA architectures requires re-deriving the schedule in terms of whatever asynchronous-load and matmul primitives those platforms expose. The high-level invariant (three operations in flight on different units) is portable; the low-level details are not.

Numerical precision and FP8. The FP8 path of FA3 reaches $\sim 1,300$ TFLOPS on H100, but FP8 attention is numerically delicate. Per-block rescaling, careful management of the running maximum, and choice of rescaling thresholds all matter for training stability. The published FA3 paper [24] and the FP8 format specification of Micikevicius et al. [18] discuss these concerns, but understanding when FP8 attention is safe in production training remains an active area of empirical research. A complementary line of work uses post-training integer quantization (`LLM.int8()` [7]) to reduce inference memory rather than training memory.

Other operators. Attention is the most prominent example of a memory-bound operator in transformers, but it is not the only one. Layer norm, MLP activations, and even sequence packing have working-set patterns that could in principle benefit from FA-style tiling and recomputation. The optimizer state itself, dominated by Adam’s [12] two moment buffers, is another large memory consumer that has received its own line of work. Inference workloads, whose

memory profile differs from training because activations need not be retained for the backward, raise a different set of optimization questions, surveyed by Pope et al. [21]. Whether the FA3 producer/consumer pipeline can be applied to all these settings, or whether it is specific to the matmul + softmax structure of attention, is an interesting design question.

5.5 Conclusion

Modern GPU memory optimization is, at the level of principle, an old story. The two strategies that dominate the space (tiling and recomputation) are direct re-applications of two of the oldest ideas in operating systems and numerical computing. Cache blocking and loop tiling, producer/consumer with a ring buffer, asynchronous I/O via DMA, demand paging, working-set sizing: all of these patterns recur, almost verbatim, inside FlashAttention, FlashAttention-3, and gradient checkpointing.

What is genuinely new is the regime. The H100 has pushed the gap between fast and slow memory to roughly 3.5×10^5 in capacity, with bandwidth and latency gaps to match. Algorithms that ignored the hierarchy on a CPU collapse on this hardware, and the hardware itself has been redesigned (TMA, WGMMMA, `mbarrier`) to support exactly the producer/consumer pattern that FA3 needs. The result is a tight loop of co-evolution: hardware exposes new primitives, software builds new algorithms on top, and hardware reads the algorithms and exposes the next generation of primitives.

For an operating systems course the lesson is that the patterns one learns for managing CPU caches, RAM pages, and producer/consumer pipelines are not historical curiosities. They are the same patterns that, at a different scale and a different granularity, are making it possible to train and serve modern large

language models on contemporary GPUs.

Table 5.1: Mapping between GPU memory-optimization techniques and classical operating systems ideas.

GPU technique	Mechanism	OS analogue
<i>Tiling strategy (working set > SRAM)</i>		
FA tiling	Compute on Q, K, V blocks fitting in SRAM	Cache blocking, loop tiling
FA3 warp specialization	Producer + consumer warps with ring buffer	Producer/consumer with bounded queue
FA3 SMEM circular buffer	2 to 3 slot ring of (K, V) tiles	Double or triple buffering for async I/O
TMA load	Async hardware transfer HBM $\rightarrow$ SMEM	DMA engine
mbarrier	Hardware semaphore on tile readiness	Semaphore, condition variable
<i>Recomputation strategy (working set > HBM)</i>		
FA backward	Drop S, P ; recompute on-chip from Q, K, V, L	Trade compute for memory at operator level
Gradient checkpointing	Store every $\sqrt{L}$ -th activation; replay segment	Demand paging at layer granularity
$\sqrt{L}$ optimum	Balance checkpoints vs. in-flight recompute	Working-set theorem (resident vs. faults)

Bibliography

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.14165.
- [2] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost, 2016. arXiv:1604.06174.
- [3] Jack Choquette. NVIDIA Hopper H100 GPU: Scaling performance. *IEEE Micro*, 43(3):9–17, 2023.
- [4] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. NVIDIA A100 tensor core GPU: Performance and innovation. *IEEE Micro*, 41(2):29–35, 2021.
- [5] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.08691, July 2023.
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.14135.

- [7] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale, 2022. arXiv:2208.07339.
- [8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*, pages 4171–4186, 2019. arXiv:1810.04805.
- [9] Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. Cache-oblivious algorithms. In *40th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 285–297, 1999.
- [10] Andreas Griewank and Andrea Walther. Algorithm 799: Revolve: An implementation of checkpointing for the reverse or adjoint mode of computational differentiation. *ACM Transactions on Mathematical Software*, 26(1):19–45, 2000.
- [11] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2203.15556.
- [12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.
- [13] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing

- activation recomputation in large transformer models. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023. arXiv:2205.05198.
- [14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.
- [15] Monica S. Lam, Edward E. Rothberg, and Michael E. Wolf. The cache performance and optimizations of blocked algorithms. In *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*, pages 63–74, 1991.
- [16] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S. Vetter. NVIDIA tensor core programmability, performance & precision. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 522–531, 2018. arXiv:1803.04014.
- [17] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.03740.
- [18] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. FP8 formats for deep learning, 2022. arXiv:2209.05433.

- [19] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax, 2018. arXiv:1805.02867.
- [20] NVIDIA Corporation. NVIDIA H100 tensor core GPU architecture. NVIDIA Whitepaper, 2022.
- [21] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023. arXiv:2211.05102.
- [22] Markus N. Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory, 2021. arXiv:2112.05682.
- [23] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC’20)*, 2020. arXiv:1910.02054.
- [24] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2407.08608.
- [25] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism, 2019. arXiv:1909.08053.
- [26] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave,

and Guillaume Lample. LLaMA: Open and efficient foundation language models, 2023. arXiv:2302.13971.

- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [28] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [29] Wm. A. Wulf and Sally A. McKee. Hitting the memory wall: Implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23(1):20–24, 1995.